Object–oriented programming

Instructor: Masoud Asghari

Web page: http://www.masses.ir/lectures/oops2017sut

Ch: 5

In this slide

- We follow:
  - https://docs.oracle.com/javase/tutorial/index.html

1. **Trail: Learning the Java Language > Annotations**

2. **Trail: Learning the Java Language > Interfaces and Inheritance**
   - **Interfaces**
   - **Inheritance**

1st Trail: Annotations

- Annotations,
  - a form of **metadata**, provide data about a program that is not part of the program itself.
  - Annotations have no direct effect on the operation of the code they annotate.
- Annotations uses
  - Information for the compiler
    - Annotations can be used by the compiler to detect errors or suppress warnings.
  - Compile-time and deployment-time processing
    - Software tools can process annotation information to generate code, XML files, and so forth.
  - Runtime processing
    - Some annotations are available to be examined at runtime

Annotations Basics

- The Format of an Annotation
  - In its simplest form, an annotation looks like the following:
    - @Entity
  - The at sign character (@) indicates to the compiler that what follows is an annotation.
  - In the following example, the annotation's name is Override:
    - @Override
      void mySuperMethod() { ... }
  - The annotation can include elements, which can be named or unnamed, and there are values for those elements:
    - @Author(
      name = "Benjamin Franklin",
      date = "3/27/2003“
)
      class MyClass() { ... }
    - Or
    - @SuppressWarnings(value = "unchecked")
      void myMethod() { ... }

Annotations Basics

- If there is just one element named value, then the name can be omitted, as in:
  - @SuppressWarnings("unchecked")
    void myMethod() { ... }
- If the annotation has no elements, then the parentheses can be omitted, as shown in the previous @Override example.
- It is also possible to use multiple annotations on the same declaration:
  - @Author(name = "Jane Doe")
    @Ebook
    class MyClass { ... }
- If the annotations have the same type, then this is called a <u>repeating annotation</u>:
  - @Author(name = "Jane Doe")
    @Author(name = "John Smith")
    class MyClass { ... }

Annotations Basics

- The annotation type
  - can be one of the types that are defined in the java.lang or java.lang.annotation packages of the Java SE API.
    - In the previous examples, *Override* and *SuppressWarnings* are predefined Java annotations.
  - It is also possible to define your own annotation type.
    - The *Author* and *Ebook* annotations in the previous example are custom annotation types.
- **Where Annotations Can Be Used?**
  - Annotations can be applied to ***declarations***:
    - Declarations of classes, fields, methods, and other program elements.
    - When used on a declaration, each annotation often appears, by convention, on its own line
  - Annotations can also be applied to the use of types *(type annotation)*.
    - *Examples:*
    - Class instance creation expression: `new @Interned MyObject();`
    - Type cast: `myString = (@NonNull String) str;`

Declaring an Annotation Type

- Many annotations replace comments in code.
- To add a metadata with an annotation, you must first define the annotation type.
  - The annotation type definition looks similar to an *interface* definition where the keyword *interface* is preceded by the at sign (@)
  - Annotation types are a form of *interface*.
  - The body of the annotation definition contains annotation type element declarations, which look a lot like methods.
    - Note that they can define optional *default* values.
  - After the annotation type is defined, you can use annotations of that type, with the values filled in

```
@interface ClassPreamble {
    String author();
    String date();
    int currentRevision() default 1;
    String lastModified() default "N/A";
    String lastModifiedBy() default "N/A";
    // Note use of array
    String[] reviewers();
}
```

```
@ClassPreamble (
    author = "John Doe",
    date = "3/17/2002",
    currentRevision = 6,
    lastModified = "4/12/2004",
    lastModifiedBy = "Jane Doe",
    // Note array notation
    reviewers = {"Alice", "Bob", "Cindy"}
)
public class Generation3List extends Generation2List {
    // class code goes here
}
```

Predefined Annotation Types

- Annotation Types Used by the Java Language
  - The predefined annotation types defined in java.lang are (@SafeVarargs, @FunctionalInterface)
  - @Deprecated
    - @Deprecated annotation indicates that the marked element is deprecated and should no longer be used.
    - The compiler generates a warning whenever a program uses a method, class, or field with the @Deprecated annotation.
  - @Override
    - @Override annotation informs the compiler that the element is meant to override an element declared in a superclass.
    - While it is not required to use this annotation when overriding a method, it helps to prevent errors. If a method marked with @Override fails to correctly override a method in one of its superclasses, the compiler generates an error.
  - @SuppressWarnings
    - @SuppressWarnings annotation tells the compiler to suppress specific warnings that it would otherwise generate.
    - Every compiler warning belongs to a category. The Java Language Specification lists two categories: *deprecation* and *unchecked*. To suppress multiple categories of warnings, use the following syntax: @SuppressWarnings({"unchecked", "deprecation"})

```
@SuppressWarnings("deprecation")
    void useDeprecatedMethod() {
        objectOne.deprecatedMethod();
    }
```

Predefined Annotation Types

- Annotations That Apply to Other Annotations
  - are called *meta-annotations*. There are several meta-annotation types defined in java.lang.annotation.
  - @Retention, @Documented, @Target, @Inherited, @Repeatable

- Type Annotations and Pluggable Type Systems
  - Before the Java SE 8 release, annotations could only be applied to declarations.
  - As of the Java SE 8 release, annotations can also be applied to any type use.
  - This means that annotations can be used anywhere you use a type.
    - A few examples of where types are used are class instance creation expressions (new), casts, implements clauses, and throws clauses.
  - This form of annotation is called a *type annotation*

- Repeating Annotations
  - The annotation type must be marked with the @Repeatable meta-annotation.

2nd **Trail:** Interfaces

- An interface
  - is a reference type, similar to a class,
  - that can contain only **constants**, **method signatures (abstract methods)**, **default methods**, **static methods**, and **nested types**.
  - Method bodies exist only for default methods and static methods.
  - Interfaces cannot be instantiated—they can only be implemented by classes or extended by other interfaces.
  - To use an interface,
    - you write a class that *implements* the interface.
    - When an instantiable class implements an interface, it provides a method body for each of the methods declared in the interface.

```java
public interface OperateCar {
    // constant declarations, if any
    // method signatures
    // An enum with values RIGHT, LEFT
    int turn(Direction direction,double radius,double startSpeed,double endSpeed);
    int changeLanes(Direction direction,double startSpeed,double endSpeed);
    int signalTurn(Direction direction,boolean signalOn);
    int getRadarFront(double distanceToCar,double speedOfCar);
    int getRadarRear(double distanceToCar,double speedOfCar);
        ......
    // more method signatures
}

public class OperateBMW760i implements OperateCar {
    // the OperateCar method signatures, with implementation --
    // for example:
    int signalTurn(Direction direction, boolean signalOn) {
        // code to turn BMW's LEFT turn indicator lights on
        // code to turn BMW's LEFT turn indicator lights off
        // code to turn BMW's RIGHT turn indicator lights on
        // code to turn BMW's RIGHT turn indicator lights off
    }

    // other members, as needed -- for example, helper classes not
    // visible to clients of the interface
}
```

Defining an Interface

- An interface declaration
  - consists of modifiers,
  - the keyword ***interface***,
  - the interface name,
  - a comma-separated list of parent interfaces (if any) after **extends**
  - and the interface body.
- An interface can extend other interfaces,
  - Unlike classes, an interface can extend any number of interfaces.
- **The Interface Body**
  - All *abstract*, *default*, and *static* **methods** in an interface are **implicitly public**, so you can omit the public modifier.
    - An abstract method within an interface is followed by a semicolon, but no braces
  - All **constant values** defined in an interface are implicitly **public, static, and final**. you can omit these modifiers.

```java
public interface InterfaceRun {
    void run(int speed);
    void stop();
}
public interface InterfaceFly {
    void fly(int height);
    void land();
}

public interface InterfaceSpaceShip extends InterfaceRun, InterfaceFly {
     // constant declarations
     double E = 2.718282; //public static final

     // method signatures
     void doSomething (int i, double x);
     int doSomethingElse(String s);
}
```

Implementing an Interface

- To declare a class that implements an interface,
  - you include an **implements** clause in the class declaration.
- Your class
  - can implement more than one interface,
  - so the **implements** keyword is followed by a comma-separated list of the interfaces implemented by the class.

```java
public class RectanglePlus implements Relatable {
    public int width = 0;
    public int height = 0;

    public RectanglePlus(int w, int h) {
        width = w;
        height = h;
    }

    public int getArea() {
        return width * height;
    }

    // a method required to implement the Relatable interface
    public int isLargerThan(Relatable other) {
        RectanglePlus otherRect = (RectanglePlus)other;
        if (this.getArea() < otherRect.getArea()) return -1;
        else if (this.getArea() > otherRect.getArea()) return 1;
        else return 0;
    }
}
```

```java
public interface Relatable {
    public int isLargerThan(Relatable other);
}
//If you know that a class implements Relatable,
//then you know that you can compare the size of
//the objects instantiated from that class.
```

```java
public class program{
    public static void main(String[] args){
        //make objects and compare them ...
    }
}
```

Using an Interface as a Type

- You can use interface names anywhere you can use any other data type name.
- If you define a **reference variable** whose type is an interface, any object you assign to it *must* be an instance of a class that implements the interface.

```
public class program {

    public static Object findLargest(Object object1, Object object2) {
        if(!(object1 instanceof Relatable && object2 instanceof Relatable))
            return null;

        Relatable obj1 = (Relatable)object1;
        Relatable obj2 = (Relatable)object2;
        if ((obj1).isLargerThan(obj2) > 0)
            return object1;
        else
            return object2;
    }

    public static void main(String[] args){
        RectanglePlus rA = new RectanglePlus(2,3);
        RectanglePlus rB = new RectanglePlus(3,3);

        RectanglePlus rL=(RectanglePlus)findLargest(rA,rB);
        System.out.println("The largest Object has "+ rL.getArea()+" size");
    }
}
```

Default Methods

- We already have

```
public interface Relatable {
    public int isLargerThan(Relatable other);
}
```

- Some people used it in their classes (implemented the interface)
- Now we want to change Relatable (add isEqual method)
- **Default methods** enable you
  - to add new functionality to the interfaces of your libraries and ensure binary compatibility with code written for older versions of those interfaces.

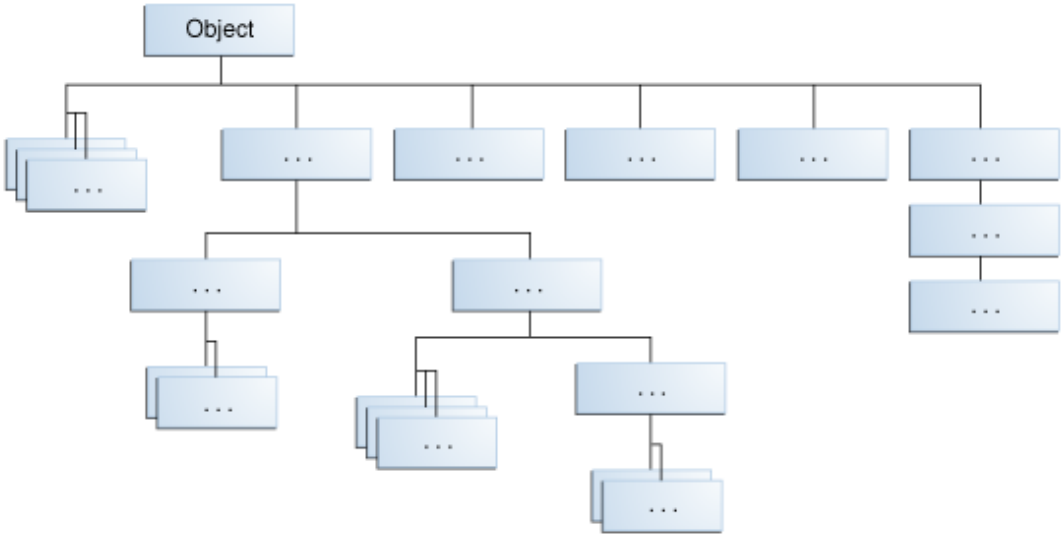
```
public interface Relatable {
    public int isLargerThan(Relatable other);

    default boolean isEqual(Relatable other){
        if(isLargerThan(other)==0)return true;
        else return false;
    }

    static int someMethod(){
        return 0;
    }
}
```

Inheritance

- A class
  - that is derived from another class is called a **subclass** (also a derived class, extended class, or child class).
- The class
  - from which the subclass is derived is called a **superclass** (also a base class or a parent class).
- Every class has one and only one direct superclass (**single inheritance**).
  - Excepting *Object*, which has no superclass,
  - In the absence of any other explicit superclass, every class is **implicitly** a subclass of Object.
- The idea:
  - When you want to create a new class and there is already a class that includes some of the code that you want,
  - You can derive your new class from the existing class. In doing this, you can reuse the fields and methods of the existing class without having to write (and debug!) them yourself.

Inheritance

- A subclass **inherits** all the **members** (**fields, methods, and nested classes**) from its superclass.
- ***Constructors*** are not members, so they are not inherited by subclasses, but the constructor of the superclass can be invoked from the subclass.
- **The Java Platform Class Hierarchy**
  - The ***Object*** class, defined in the ***java.lang*** package, defines and implements behavior common to all classes
  - Many classes derive directly from *Object*, other classes derive from some of those classes, and so on, forming a **hierarchy of classes.**



An Example of Inheritance (inherit_1)

```java
public class Bicycle {
    // the Bicycle class has three fields
    public int cadence;
    public int gear;
    public int speed;

    // the Bicycle class has one constructor
    public Bicycle(int startCadence, int startSpeed, int startGear) {
        gear = startGear;
        cadence = startCadence;
        speed = startSpeed;
    }

    // the Bicycle class has four methods
    public void setCadence(int newValue) {
        cadence = newValue;
    }

    public void setGear(int newValue) {
        gear = newValue;
    }

    public void applyBrake(int decrement) {
        speed -= decrement;
    }

    public void speedUp(int increment) {
        speed += increment;
    }
}
```

An Example of Inheritance (inherit_1)

```java
public class MountainBike extends Bicycle {

    // the MountainBike subclass adds one field
    public int seatHeight;

    // the MountainBike subclass has one constructor
    public MountainBike(int startHeight,int startCadence,int startSpeed,int startGear){
        super(startCadence, startSpeed, startGear);
        seatHeight = startHeight;
    }

    // the MountainBike subclass adds one method
    public void setHeight(int newValue) {
        seatHeight = newValue;
    }
}
```

- *MountainBike* inherits all the fields and methods of Bicycle
- and adds the field *seatHeight* and a method to set it.
- Except for the ***constructor***, it is like a writing a new *MountainBike* class, with <u>four fields and five methods</u>.

Inheritance

- A subclass **inherits**
  - all of the **public** and **protected** members of its parent, no matter what package the subclass is in.
  - If the subclass is in the same package as its parent, it also inherits the **package-private** members of the parent.
- What you can do in a **subclass**
  1. The **inherited fields** can be used directly, just like any other **fields**.
  2. You can declare a **field** in the subclass with the same name as the one in the superclass, thus **hiding** it.
  3. You can declare **new fields** in the subclass that are not in the superclass.
  4. The **inherited methods** can be used directly as they are.
  5. You can write a **new *instance* *method*** in the subclass that has the same signature as the one in the superclass, thus **overriding** it.
  6. You can write a **new *static* *method*** in the subclass that has the same signature as the one in the superclass, thus **hiding** it.
  7. You can declare **new methods** in the subclass that are not in the superclass.
  8. You can write a **subclass constructor** that invokes the **constructor** of the superclass, either **implicitly** or by using the keyword **super**.

Inheritance

- **Private Members in a Superclass**
  - A subclass does not inherit the private members of its parent class.
  - However, if the superclass has public or protected methods for accessing its private fields, these can also be used by the subclass.

- **Casting Objects**

```
public MountainBike myBike = new MountainBike();
```

  - myBike is of type MountainBike.
  - *MountainBike* is descended from *Bicycle* and *Object*.
    - Therefore, a *MountainBike* is a *Bicycle* and is also an *Object*,
    - and it can be used wherever *Bicycle* or *Object* objects are called for.
  - Reverse is not always true
    - an *Object* may be a *Bicycle* or a *MountainBike*, but it isn't necessarily.
- Casting shows the use of an object of one type in place of another type,
  - among the objects permitted by inheritance and implementations.

Objects casting

- if we write
  - **Object** obj = new **MountainBike**();
  - then obj is both an **Object** and a **MountainBike**
  - (until such time as obj is assigned another object that is not a MountainBike).
  - This is called *implicit casting.*
- If then, we write
  - **MountainBike** myBike = obj;
  - we would get a compile-time error
  - because obj is not known to the compiler to be a MountainBike.
- However,
  - we can tell the compiler that we promise to assign a MountainBike to obj
  - by *explicit casting*
  - **MountainBike** myBike = (**MountainBike**)obj;
  - If obj is not a MountainBike at runtime, an *exception* will be thrown.
  - To avoid this, make sure type of the object by using *instanceof*
  - if (obj *instanceof* **MountainBike**) **MountainBike** myBike = (**MountainBike**)obj;

Multiple Inheritance of State, Implementation, and Type

- ***multiple inheritance of state***
  - is the ability to inherit **fields** from multiple classes.
  - to avoid this issue, Java does not permit you to extend more than one class
  - Because interfaces do not contain fields, you do not have to worry about problems that result from multiple inheritance of state.
- ***multiple inheritance of implementation***
  - is the ability to inherit **method definitions** from multiple classes.
  - *Default and static methods* introduce one form of multiple inheritance of implementation. *Note: Static methods in interfaces are never inherited.*
  - A class can implement more than one interface, which can contain default methods that have the same name.
  - In this case, the compiler or the user must decide which one to use.
- ***multiple inheritance of type***
  - is the ability of a class to implement more than one interface.
  - An object can have multiple types:
    - the type of its own class and the types of all the interfaces that the class implements.
  - This means that if a variable is declared to be the type of an interface, then its value can reference any object that is instantiated from any class that implements the interface.

Overriding and Hiding Methods

- Instance Methods
  - An instance method in a subclass with the <u>same signature and return type</u> as an instance method in the superclass ***<u>overrides</u>*** the superclass's method.
  - When overriding a method, you might want to use the @Override annotation.
  - that instructs the compiler that you intend to override a method in the superclass.

- Static Methods
  - If a subclass defines a static method with the same signature as a static method in the superclass,
  - then the method in the subclass ***<u>hides</u>*** the one in the superclass.

```java
public class Animal {
    public static void testClassMethod() {
        System.out.println("The static method in Animal");
    }
    public void testInstanceMethod() {
        System.out.println("The instance method in Animal");
    }
}
```

```java
public class Cat extends Animal {
    public static void testClassMethod() {
        System.out.println("The static method in Cat");
    }
    public void testInstanceMethod() {
        System.out.println("The instance method in Cat");
    }

    public static void main(String[] args) {
        Cat myCat = new Cat();
        Animal myAnimal = myCat;

        myCat.testInstanceMethod();
        myAnimal.testInstanceMethod();
        System.out.println("-----");

        Animal.testClassMethod();
        Cat.testClassMethod();
        System.out.println("-----");

        myCat.testClassMethod();
        myAnimal.testClassMethod();
    }
}
```

```
The instance method in Cat
The instance method in Cat
-----
The static method in Animal
The static method in Cat
-----
The static method in Cat
The static method in Animal
```

Overriding and Hiding Methods

- **Default methods in <u>interfaces</u>**
  - are inherited like instance methods.
  - When the supertypes of a class or interface provide multiple default methods with the same signature, the Java compiler follows inheritance rules to resolve the **name conflict**.
    - Instance methods are preferred over interface default methods.
    - Methods that are already overridden by other candidates are ignored. This circumstance can arise when supertypes share a common ancestor.
    - If two or more independently defined default methods conflict, or a default method conflicts with an abstract method, then the Java compiler produces a compiler error. You must explicitly override the supertype methods.
- Note: **Static methods in <u>interfaces</u>** are never inherited.
- **<u>Modifiers</u>**
  - The access specifier for an overriding method can allow more, but not less, access than the overridden method.
  - You will get a compile-time error if you attempt to change an instance method in the superclass to a static method in the subclass, and vice versa.

Polymorphism

- The Java virtual machine (JVM) calls the appropriate method for the object that is referred to in each variable. It does **not** call the method that is defined by the variable's type.
- This behavior is referred to as *virtual method invocation* and demonstrates an aspect of the important polymorphism features in the Java language.

```java
public class Bicycle {
    //...
    public void printDescription(){
        System.out.println("\nBike is " + "in gear " + this.gear + " with a cadence of "
        + this.cadence + " and travelling at a speed of " + this.speed + ". ");
    }
}
//-----------------------------------------------------------------
public class MountainBike extends Bicycle {
    private String suspension;

    public MountainBike(int startCadence,int startSpeed,int startGear,String suspensionType){
        super(startCadence,startSpeed,startGear);
        this.setSuspension(suspensionType);
    }

    public String getSuspension(){
      return this.suspension;
    }

    public void setSuspension(String suspensionType) {
        this.suspension = suspensionType;
    }

    public void printDescription() {
        super.printDescription();
        System.out.println("The " + "MountainBike has a" + getSuspension() + " suspension.");
    }
}
```

Polymorphism

```java
public class RoadBike extends Bicycle{
    private int tireWidth;
    public RoadBike(int startCadence,int startSpeed,int startGear,int newTireWidth){
        super(startCadence,startSpeed,startGear);
        this.setTireWidth(newTireWidth);
    }

    public int getTireWidth(){
      return this.tireWidth;
    }
    public void setTireWidth(int newTireWidth){
        this.tireWidth = newTireWidth;
    }
    public void printDescription(){
        super.printDescription();
        System.out.println("The RoadBike" + " has " + getTireWidth() +" MM tires.");
    }
}
//-----------------------------------------------------------------
public class TestBikes {
  public static void main(String[] args){
    Bicycle bike01, bike02, bike03;

    bike01 = new Bicycle(20, 10, 1);
    bike02 = new MountainBike(20, 10, 5, "Dual");
    bike03 = new RoadBike(40, 20, 8, 23);

    bike01.printDescription();
    bike02.printDescription();
    bike03.printDescription();
  }
}
```

Output

```
Bike is in gear 1 with a cadence of 20 and travelling at a speed of 10.

Bike is in gear 5 with a cadence of 20 and travelling at a speed of 10.
The MountainBike has a Dual suspension.

Bike is in gear 8 with a cadence of 40 and travelling at a speed of 20.
The RoadBike has 23 MM tires.
```

Using the Keyword **super**

- Accessing Superclass Members
  - If your method **overrides** one of its superclass's methods, you can invoke the overridden method through the use of the keyword ***super***.
  - You can also use super to refer to a ***hidden field***

```
public class Superclass {

    public void printMethod() {
        System.out.println("Printed in Superclass.");
    }
}

public class Subclass extends Superclass {

    // overrides printMethod in Superclass
    public void printMethod() {
        super.printMethod();
        System.out.println("Printed in Subclass");
    }
    public static void main(String[] args) {
        Subclass s = new Subclass();
        s.printMethod();
    }
}
```

Output

```
Printed in Superclass.
Printed in Subclass
```

Using the Keyword **super**

- Use the *super* keyword to invoke *a superclass's constructor*.
  - Invocation of a superclass constructor must be the first line in the subclass constructor.
  - The syntax for calling a superclass constructor is
    - super();
    - super(parameter list);
  - With super(), the superclass no-argument constructor is called.
  - With super(parameter list), the superclass constructor with a matching parameter list is called.
- Note:
  - If a constructor does not explicitly invoke a superclass constructor, the Java compiler automatically inserts a call to the no-argument constructor of the superclass.
  - If the super class does not have a no-argument constructor, you will get a compile-time error.
  - If a subclass constructor invokes a constructor of its superclass,
    - either explicitly or implicitly,
    - there will be a chain of constructors called,
    - all the way back to the constructor of Object.
    - It is called **constructor chaining**

Object as a Superclass

- The *Object* class, in the java.lang package,
  - Every class is a descendant, direct or indirect, of the Object class.
  - Every class you use or write inherits the instance methods of Object.
  - You may need to override them with code that is specific to your class.
- The methods inherited from *Object*

- `protected Object clone() throws CloneNotSupportedException`
  Creates and returns a copy of this object.
- `public boolean equals(Object obj)`
  Indicates whether some other object is "equal to" this one.
- `protected void finalize() throws Throwable`
  Called by the garbage collector on an object when garbage collection determines that there are no more references to the object
- `public final Class getClass()`
  Returns the runtime class of an object.
- `public int hashCode()`
  Returns a hash code value for the object.
- `public String toString()`
  Returns a string representation of the object.

Object as a Superclass

- **The *equals()* Method**
  - The equals() method compares two objects for equality and returns true if they are equal.
  - The equals() method provided in the Object class uses the identity operator (==) to determine whether two objects are equal.
  - The equals() method provided by *Object* tests whether the object references are equal—that is, if the objects compared are the exact same object.
  - To test whether two objects are equal in the sense of equivalency (containing the same information), you must **override** the equals() method.

```
public class Book {
    ...
    public boolean equals(Object obj) {
        if (obj instanceof Book)
            return ISBN.equals((Book)obj.getISBN());
        else
            return false;
    }
}
```

Object as a Superclass

- **The *finalize()* Method**
  - The *Object* class provides a callback method, *finalize(),* that may be invoked on an object when it becomes garbage.
  - Object's implementation of *finalize()* does nothing—you can override *finalize()* to do cleanup, such as freeing resources.
- **The *getClass()* Method**
  - **You cannot override getClass.** It is finial method
  - The *getClass()* method returns a *Class* object,
  - which has methods you can use to get information about the class,
  - such as its name *getSimpleName()*, its superclass *getSuperclass(),* and the interfaces it implements *getInterfaces().*

```
System.out.println("The object's" + " class is " + obj.getClass().getSimpleName());
```

  - The *Class* class, in the java.lang package, has a large number of methods (more than 50).

Object as a Superclass

- **The *hashCode()* Method**
  - The value returned by *hashCode()* is the object's hash code,
  - which is the object's memory address in hexadecimal.

- **The *toString()* Method**
  - The Object's *toString()* method returns a String representation of the object.
  - The String representation for an object depends entirely on the object,
  - which is why you need to override toString() in your classes.

Final Classes and Methods

- You can declare some or all of a class's methods final.
  - You use the **final** keyword in a method declaration to indicate that the method cannot be overridden by subclasses.
- You might wish to make a method final
  - if it has an implementation that should not be changed and it is critical to the consistent state of the object.
  - Methods called from constructors should generally be declared final.
    - If a constructor calls a non-final method, a subclass may redefine that method with surprising or undesirable results.

- You can also declare an **entire class final**.
  - A class that is declared final cannot be subclassed.

Abstract Methods and Classes

- An **abstract class**
  - is a class that is declared abstract
    - it may or may not include abstract methods.
  - Abstract classes cannot be instantiated, but they can be subclassed.

- An **abstract method**
  - is a method that is declared without an implementation
  - (without braces, and followed by a semicolon)
  - Note: Methods declared in interfaces are *implicitly* abstract
  - **If a class includes abstract methods, then the class itself must be declared abstract**

```
public abstract class GraphicObject {
   protected int x,y;

   abstract void draw();
   abstract void moveTo(double deltaX, double deltaY);

   public void notAbstract(){
      System.out.println("not an abstract method");
   }
}
```

- When an abstract class is subclassed,
  - the subclass usually provides implementations for all of the abstract methods in its parent class.
  - However, if it does not, then the subclass must also be declared abstract.

Abstract Methods and Classes

- **Abstract Classes Compared to Interfaces**
  - Abstract classes are similar to interfaces.
  - You cannot instantiate them, and they may contain a mix of methods declared with or without an implementation.
  - However, with abstract classes, you can declare fields that are not static and final, and define public, protected, and private concrete methods.
  - With interfaces, all fields are automatically public, static, and final, and all methods that you declare or define (as default methods) are public.
  - In addition, you can extend only one class, whether or not it is abstract, whereas you can implement any number of interfaces.
- **When an Abstract Class Implements an Interface**
  - In the section on Interfaces, it was noted that a class that implements an interface must implement all of the interface's methods.
  - However, It is possible, to define a **class** that <u>does not implement all of the interface's methods</u>, provided that the **class is declared to be abstract**.
- **Class Members**
  - An abstract class may have static fields and static methods.
  - You can use static members with a class reference as you would with any other class.

Exercises: Interfaces and Inheritance

- Write an interface for TV (TvInterface)
  - Includes get Power,Volume,Channel and set Power,Volume,Channel
- Extend TvInterface for Smart TV (SmartTvInterface)
  - Includes getOnlineStatus
- Write abstract class *SmartTVClass* that implements SmartTvInterface without implementing channel functionalities
- Write class *SmartAnalogTVClass* that extends *SmartTVClass and* implements channel functionalities
- Write class *SmartDigTVClass* that extends *SmartTVClass and* implements channel functionalities
- Add new default method for SmartTvInterface
  - to include getFullStatus which returns an string containing Power,Volume,Channel status
  - Do not touch you class implementation